

02_Jak_stworzyc_od_zera

Jak stworzyc taki projekt od zera

Ten dokument opisuje proces stworzenia podobnego projektu w Unity krok po kroku. Nie jest to tylko instrukcja uruchomienia gotowej paczki, ale opis jak samodzielnie zbudowac podobna symulacje.

1. Przygotowanie projektu Unity

1. Otworz Unity Hub.
2. Utworz nowy projekt typu Universal 3D.
3. Nazwij projekt np. Automatic parking.
4. Po otwarciu Unity utworz foldery:

[blok kodu / diagram w pliku Markdown]

Minimalnie potrzebne sa:

- Scripts/Parking na skrypty runtime,
- Editor na generator scen,
- Scenes na sceny,
- Materials na materialy.

2. Podstawowe obiekty w Unity

Projekt mozna zrobic bez gotowych assetow, tylko z prymitywow Unity:

- Cube jako droga,
- Cube jako samochod,
- Cube jako kabina,
- Cylinder jako kola,
- Cube jako linie parkingowe,
- Cube jako przeszkody i sciany.

To wystarczy do zaliczenia, bo najwazniejsza jest logika.

3. Stworzenie typow danych

Pierwszy plik to ParkingTypes.cs.

Nalezy zdefiniowac:

- enum stanow FSM,
- enum typu parkowania,
- strukture odczytow sensorow,

- strukture kandydata miejsca.

Przykład stanów:

[blok kodu / diagram w pliku Markdown]

Dodatkowy stan:

[blok kodu / diagram w pliku Markdown]

Ta separacja jest ważna, bo łatwiej potem tłumaczyć kod.

4. Stworzenie kontrolera auta

Plik: ParkingCarController.cs

Kontroler powinien mieć pola:

- wheelBase,
- carLength,
- carWidth,
- maxForwardSpeed,
- maxReverseSpeed,
- acceleration,
- brakeAcceleration,
- maxSteeringAngle,
- steeringRate.

Minimalne metody:

[blok kodu / diagram w pliku Markdown]

Tryb pierwszy: kinematyczny

W prostym trybie auto jedzie tak:

1. Oblicz prędkość docelową z gazu.
2. Płynnie zmieniaj aktualną prędkość.
3. Płynnie zmieniaj kąt skrętu.
4. Oblicz prędkość obrotową:

[blok kodu / diagram w pliku Markdown]

5. Przesun auto do przodu:

[blok kodu / diagram w pliku Markdown]

Tryb drugi: ruch po punktach

Do prezentacji najlepiej uzyc ruchu po punktach:

1. Zapamietaj pozycje startowa odcinka.
2. Zapamietaj kat startowy.
3. Oblicz roznice kata przez `Mathf.DeltaAngle`.
4. Przesuwaj auto przez `Vector3.Lerp`.
5. Obracaj auto przez lagodne przejście `SmoothStep`.

To daje stabilny pokaz bez losowej fizyki.

5. Stworzenie sensorow

Plik: `ParkingSensors.cs`

Sensory powinny mierzyc:

- przod,
- tyl,
- bok prawy: przod/srodek/tyl,
- bok lewy: przod/srodek/tyl.

W Unity robi sie to przez:

[blok kodu / diagram w pliku Markdown]

Warto rysowac promienie:

[blok kodu / diagram w pliku Markdown]

Kolory:

- zielony: wolne,
- czerwony: przeszkoda.

6. Wykrywanie miejsca

Prosta logika:

1. Jeśli trzy sensory boczne pokazują dużą odległość, zaczyna się luka.
2. Zapamiętaj pozycję początku luki.
3. Gdy auto jedzie dalej, aktualizuj koniec luki.
4. Jeśli luka jest dłuższa niż wymagane minimum, sprawdź objętość CheckBox.
5. Jeśli CheckBox nie wykrywa przeszkody, miejsce jest poprawne.

Dla parkowania równoległego wymagaj dłuższej luki, np.:

[blok kodu / diagram w pliku Markdown]

Dla prostopadłego:

[blok kodu / diagram w pliku Markdown]

7. Stworzenie FSM

Plik: ParkingStateMachine.cs

Najprostszy wzorzec:

[blok kodu / diagram w pliku Markdown]

W każdym stanie:

- wykonaj logikę,
- sprawdź warunek przejścia,
- jeśli warunek spełniony, wywołaj `ChangeState(next)`.

Metoda `ChangeState` powinna:

- ustawić nowy stan,
- wyzerować licznik czasu stanu,
- wykonać akcje specjalne, np. zatrzymanie w `Parked`.

8. Dodanie `EmergencyStop`

Stan awaryjny powinien działać z każdego etapu manewru.

Przykład:

[blok kodu / diagram w pliku Markdown]

W projekcie przeszkoda dynamiczna może też wywołać:

[blok kodu / diagram w pliku Markdown]

To pozwala pokazać reakcje nawet wtedy, gdy fizyczne kolizje są uproszczone.

9. Stworzenie scenariusza manewru

Plik: ParkingScenario.cs

Najprostszy sposób:

1. Utwórz listę punktów docelowych.
2. Każdy punkt ma:
 - pozycję,
 - kąt,
 - prędkość,
 - informację czy jedziemy do przodu czy cofamy,
 - stan FSM, który ma być wyświetlany.
3. FSM wykonuje punkty po kolei.

Przykład:

[blok kodu / diagram w pliku Markdown]

Im więcej małych punktów, tym płynniej wygląda ruch.

10. Jak stroić punkty parkowania

Najpraktyczniejsza metoda:

1. Ustaw auto przed miejscem.
2. Dodaj punkt pozycjonowania.
3. Dodaj punkt wejścia w miejsce.
4. Dodaj punkt połowy.
5. Dodaj punkt końcowy.
6. Uruchom Play.

7. Jesli auto zahacza:

- odsun punkty od przeszkody,
- zmniejsz skok kata miedzy punktami,
- dodaj punkt posredni.

8. Jesli auto stoi na linii:

- przesun finalny punkt do srodka miejsca.

Dla miejsca miedzy liniami zA i zB srodek to:

[blok kodu / diagram w pliku Markdown]

Przyklad z mapy 3:

[blok kodu / diagram w pliku Markdown]

11. Generator scen

Zamiast recznie klikac wszystko w Unity, mozna napisac skrypt edytorowy:

[blok kodu / diagram w pliku Markdown]

Taki generator:

- tworzy materialy,
- tworzy auta,
- tworzy drogi,
- tworzy linie,
- ustawia UI,
- ustawia kamere,
- zapisuje sceny.

Zaletą: jesli cos sie zepsuje, mozna jednym kliknieciem odbudowac caly projekt.

12. Stworzenie modelu auta

Minimalny model:

1. Body - cube, skala np. 2.0 x 0.72 x 4.45.
2. Cabin - mniejszy cube na gorze.
3. FrontMarker - maly pasek z przodu.
4. Cztery kola - cylindry.

Do auta dodaj:

- RigidBody jako kinematic,
- ParkingCarController,
- ParkingSensors,
- ParkingStateMachine,
- ParkingManeuverPlanner,
- VehicleVisualRig.

13. Stworzenie mapy 1

Mapa 1:

- droga na srodku,
- parking po prawej,
- wiekszosc miejsc zajeta,
- jedna luka za waska,
- jedna luka docelowa.

Cel: pokazac, ze auto nie parkuje w pierwszym lepszym miejscu.

14. Stworzenie mapy 2

Mapa 2:

- ulica,
- auta zaparkowane rownolegle,
- miejsce docelowe,
- manewr cofania.

Cel: pokazac parkowanie rownolegle.

15. Stworzenie mapy 3

Mapa 3:

- parking po lewej,
- czerwone auto z naprzeciwka,
- niebieskie auto czeka,
- po przejechaniu czerwonego auta parkuje.

Cel: pokazac stan EmergencyStop.

16. Dodanie HUD

HUD powinien pokazywac:

- aktualny stan,

- predkosc,
- kat skretu,
- odczyty sensorow,
- czy jest kandydat miejsca.

To bardzo pomaga podczas obrony.

17. Testowanie

Testuj w tej kolejnosci:

1. Czy sceny sie generuja.
2. Czy menu laduje mapy.
3. Czy auto rusza.
4. Czy HUD pokazuje zmiane stanow.
5. Czy auto konczy w Parked.
6. Czy mapa 3 przechodzi przez EmergencyStop.
7. Czy finalne pozycje aut nie zahaczaja innych aut.

18. Build

Po wygenerowaniu scen:

1. Otworz File -> Build Profiles albo File -> Build Settings.
2. Sprawdz, czy sceny sa w liscie:
 - MainMenu,
 - Map_01_Perpendicular,
 - Map_02_Parallel,
 - Map_03_Dynamic.
3. Wybierz Windows.
4. Zbuduj .exe.

19. Co nagrać na filmie

Film 2-3 min:

1. Menu.
2. Mapa 1: auto parkuje prostopadle.
3. Mapa 2: auto parkuje rownolegle.
4. Mapa 3: czerwone auto wymusza zatrzymanie, potem niebieskie parkuje po lewej.
5. Krotko pokaz Scene View z promieniami sensorow.

20. Jak tłumaczyć uproszczenia

Najlepsze wyjaśnienie:

"Projekt skupia się na algorytmie sterowania, a nie na zaawansowanej fizyce pojazdu. Dlatego zastosowano uproszczony model kinematyczny i deterministyczny tor parkowania po punktach.

Dzięki temu wynik jest powtarzalny, a w projekcie widac sensory, FSM i reakcje na przeszkody."